

# Assignment 1: Basic Fitness Tracker App Structure

**Release Date:** September 14, 2025

**Due Date:** Sunday, September 28, 2025, 11:59 PM EST

**Weight:** 12.5% of final grade

## Objective

Build a React Native app using Expo and TypeScript that allows users to log fitness activities (e.g., running, swimming) with fields for activity type, duration, and calories burned. The app will display a list of logged activities using `FlatList`, allow users to delete activities, and update existing activities. This assignment focuses on using core React Native components, managing state with `useState`, and handling user input with TypeScript, providing a foundation for mobile app development.

## Technologies

- **React Native:** For building the mobile app UI and functionality.
- **TypeScript:** For type-safe development of the app.
- **Expo CLI:** For project setup, development, and testing via Expo Go or emulators.
- **Node.js (v22):** Required for Expo CLI and project dependencies.

## Assignment Requirements

Build a **basic fitness tracker app** using React Native, TypeScript, and Expo that allows users to log, view, update, and delete fitness activities. The app manages activities in memory using `useState` and does not require persistent storage or deployment.

### 1 React Native App

- **Setup:** Download the starter project by [clicking this link](#) to obtain `assignment-1.tar.gz` containing `package.json`, `package-lock.json` and starter code for `App.tsx`. Extract the `.tar.gz` file, navigate to the project directory, run `npm install` to install dependencies, and edit `App.tsx` to implement the required functionality.
- **Storage:** Store activities in an in-memory array managed by `useState`, where each activity object must have:
  - `id` (string, unique, use a timestamp: `Date.now().toString()`).

- `type` (string, required, e.g., "Running", "Swimming").
- `duration` (number, required, minutes, positive integer).
- `calories` (number, optional input, positive integer, default to `duration * 10` if not provided).

In `types.ts` file, define a TypeScript type named `Activity` with the following properties:

```
{
  id: string;
  type: string;
  duration: number;
  calories: number;
}
```

Export the type to make it available for use. In `App.tsx`, import the `Activity` type using:

```
import { Activity } from "../types";
```

Example data:

```
[
  {
    "id": "1634567890123",
    "type": "Running",
    "duration": 30,
    "calories": 300
  },
  {
    "id": "1634567890456",
    "type": "Swimming",
    "duration": 45,
    "calories": 500
  }
]
```

- **Components:** Use core React Native components: `View`, `Text`, `TextInput`, `Button`, `FlatList`. Use `useState` to manage the activity array, form inputs, and editing state with TypeScript types.

## 2 Add Activity

- **Screen:** Implement a form in `App.tsx` to log activities with:
  - A `TextInput` for activity type (e.g., "Running", "Swimming")
  - A `TextInput` for duration (positive integers in minutes)
  - A `TextInput` for calories (optional, positive integer if provided)
  - A `Button` to submit the form (labeled `Add Activity`)
- **User Input:**
  - Validate inputs:

- If `type` is empty (after trimming whitespace), show an `Alert` with title `Error` and message `Please enter an activity type`
- If `duration` is missing, non-numeric, or non-positive (e.g., `0`), show an `Alert` with title `Error` and message `Duration must be a positive integer`
- If `calories` is provided but is non-numeric, or non-positive, show an `Alert` with title `Error` and message `Calories must be a positive integer`
- On submit, create a new activity with `id` (timestamp), `type`, `duration`, and `calories` (use provided value if entered, otherwise `duration * 10`), add it to the array, and **clear the form**.
- **Example:**
  - Input `type: "Cycling", duration: 20` (calories empty) adds `{ id: <unique>, type: "Cycling", duration: 20, calories: 200 }` to the array.
  - Input `type: "Cycling", duration: 20, calories: 250` adds `{ id: <unique>, type: "Cycling", duration: 20, calories: 250 }`.

### 3 Update Activity

- **Screen:** Reuse the form in `App.tsx` to update existing activities.
- **User Input:**
  - Provide an edit button (labeled `Edit`) for each activity in the `FlatList`.
  - On clicking `Edit`, pre-fill the form with the activity's `type`, `duration`, and `calories`, and change the submit button label to `Update Activity`.
  - Validate inputs (same as [Add Activity](#)): non-empty `type`, positive integer `duration`, and if `calories` provided, positive integer `calories`, with exact error messages.
  - On submit, update the activity in the array by `id` with new `type`, `duration`, and `calories` (use provided value if entered, otherwise `duration * 10`), then clear the form and revert to add mode (change the submit button label to `Add Activity`).
- **Example:**
  - Edit an activity `{ id: "123", type: "Running", duration: 30, calories: 300 }` to type: `"Swimming", duration: 40` (calories empty), updating it to `{ id: "123", type: "Swimming", duration: 40, calories: 400 }`.
  - If calories changed to 450, updates to `{ id: "123", type: "Swimming", duration: 40, calories: 450 }`.

### 4 Delete Activity

- **Screen:** Provide a delete button (labeled `Delete`) for each activity in the `FlatList`.
- **User Input:**
  - On clicking `Delete`, remove the activity by `id` from the array.
  - If the deleted activity is being edited, clear the form and revert to add mode (change the submit button label to `Add Activity`).

## 5 Display Activities

- **Screen:** Use a `FlatList` in `App.tsx` to display all logged activities.
- **UI Requirements:**
  - Each activity shows its `type`, `duration`, and `calories` (e.g., "Running - 30 min - 300 cal").
  - Each activity includes functional edit and delete buttons.
  - Use the provided `StyleSheet` for consistent styling.

### Note

Handle only the specified errors (empty `type`, empty `duration`, invalid `duration`, invalid `calories` if provided).

For invalid `duration` or `calories`, the autograder will only test:

- Non-numeric strings (e.g., "abc" )
- Zero (e.g., 0 )
- Negative numbers (e.g., -5 )
- Decimals (e.g., 1.23 )

Error messages must match exactly as specified ( Please enter an activity type , Duration must be a positive integer , Calories must be a positive integer ) to pass the autograder.

Activities are stored in memory with `useState` and will reset on app restart.

## Testing Your Implementation

### Functionality Checklist

- ☐ Form allows logging and updating activities with `type` (via `TextInput` ), `duration`, and optional `calories` .
- ☐ `FlatList` displays all activities with `type`, `duration`, `calories`, an `Edit` button, and a `Delete` button.
- ☐ Edit button pre-fills the form with the activity's data and updates the activity on submission.
- ☐ Delete button removes the correct activity from the list.
- ☐ Input validation prevents empty `type`, invalid `duration`, or invalid `calories` (if provided) with exact error messages.
- ☐ App runs locally via Expo Go or an emulator.

### Manual Testing

1. Start your app locally from your project directory:

```
npx expo start
```

2. Test your app using:

- **Expo Go:** Open the Expo Go app on your iOS/Android device and scan the QR code.
- **Emulator:** Press `i` to launch the iOS simulator, or `a` to launch the Android emulator.

## Sample Test Cases

To verify that your fitness tracker app meets the assignment requirements, use the provided test script `a1.sample.test.js`. The autograder will run similar tests when evaluating your submission. This script checks core functionality — adding, updating, deleting, and displaying activities — as well as input validation with exact error messages. The test cases in this script correspond to 80% of the assignment grade used by the autograder.

1. [Click](#) to download `a1.sample.test.js`, and save it to your project directory.

2. Execute the test script:

```
npm test
```

Review the output to identify and fix any failing tests. Use Expo Go or an emulator to manually verify UI behavior alongside the automated tests.

### Tip

Test thoroughly in Expo Go or an emulator to ensure mobile compatibility.

Use `console.log` or `Alert` to debug state changes.

Ensure TypeScript types are correctly defined and used.

Use the provided `StyleSheet` for consistent UI.

Ensure error messages match exactly to pass the autograder.

## Submission Instructions

### 1 Generate the Archive

From the parent directory of your project, create a `.tar.gz` archive with:

```
cd /path/to/parent/directory
tar zcvf 1234567890.tar.gz assignment-1
```

- Replace `assignment-1` with the name of your project directory.
- Replace `1234567890` with your student number.
- The archive must contain **exactly one top-level folder** (your project directory) with the required files.

- **Do not** include `node_modules/` or other generated files (e.g., `.expo/` ).

Your project directory (e.g., `assignment-1` ) should **exactly** be:

```
assignment-1
├── app.json
├── App.tsx
├── assets
├── index.ts
├── package-lock.json
├── package.json
├── tsconfig.json
└── types.ts
```

⚠ Any submission that fails to meet the specified format or structure and requires TA intervention for the autograder to work will receive a **20-point deduction**.

## 2 Verify Submission Structure

To avoid a 20-point deduction, use the provided Python script to check your `.tar.gz` file before submission.

1. [Download `check\_submission\_structure.py`](#) and save it in the parent directory of your project.
2. Ensure you have Python installed. Check by running `python --version` in your terminal/command prompt. If not installed, download it from [python.org](https://python.org) <sup>↗</sup>.
3. From the parent directory of your project, run:

```
python check_submission_structure.py 1234567890.tar.gz
```

- Replace `1234567890` with your student number.

## 3 Submit to Quercus

Submit your `.tar.gz` file to [Quercus](#) <sup>↗</sup>.

### Note

You are allowed to submit unlimited times. Only your latest submission before the deadline will be graded. Quercus automatically appends a suffix to the file name after the first submission — this will not affect grading.

## Grading Scheme (100 Points)

Your assignment will be graded automatically using an autograder, which will run your app via `npx expo start` and test functionality through predefined test cases. Partial marks will be awarded for partial functionality.

## Add Activity (52 Points)

- **Add button present** (2 pts)
- **Valid adds (14 pts):**
  - Running, 20 min → 200 cal (10 pts)
  - Walking, 40 min, 288 cal (4 pts)
- **Error handling (33 pts):** Autograder tests these invalid inputs:
  - Empty/whitespace `type`
  - Empty, non-numeric, 0, negative, or decimal `duration`
  - Non-numeric, 0, negative, or decimal `calories`

Correct error message required for each.

- **Form clears after add** (3 pts)

## Update Activity (33 Points)

- **Prefill form + "Update Activity" button** (5 pts)
- **Valid updates (14 pts):** update `type`, `duration`, or `calories` individually, while other fields remain unchanged.
- **Error handling (14 pts):** same invalid cases as in Add.

## Display Activities (6 points)

- Show activities in list with correct `type`, `duration`, `calories` (2 pts)
- Each item has working **Edit** (2 pts) and **Delete** (2 pts) buttons

## Delete Activity (9 Points)

- Remove activity not being edited (3 pts)
- Remove activity while editing: clears form + updates list (6 pts)

## Resources

---

- **Course Materials:**
  - [Lecture 1: Course Introduction and React Refresher](#)
  - [Lecture 2: React Native Fundamentals](#)
- **External Resources:**

- [React Native Documentation](#) ↗
- [Expo Documentation](#) ↗
- [TypeScript Documentation](#) ↗
- [TypeScript with React Native](#) ↗

## Questions?

---

### 1. Discussion Board:

- Post questions on [course discussion board](#) ↗
- Search existing discussions first
- Use clear titles and provide relevant code snippets

### 2. Office Hours:

- Time: Mondays, 2:00 PM — 3:00 PM
- Location: Room 7206, Bahen Centre for Information Technology

### 3. Tips for Getting Help:

- Start early to allow time for questions
- Be specific about your problem
- Share what you've tried
- Include relevant error messages

Last updated on October 18, 2025